

An introduction to R

Course website: <https://chamilo.unige.ch/home/courses/S411001/>

1 Introduction

R is a powerful statistical software which has numerous advantages over other packages, such as SPSS or Stata, the main ones being that it is totally cross-platform (you can install it on Microsoft Windows, Mac OS and most Linux distributions) and that it is open-source (it is free). See the official webpage <http://www.r-project.org> for general information; you can download the software and contributed packages at <http://stat.ethz.ch/CRAN/>.

Editors like **Rstudio** are also available for installation and become practical for everyday interaction with the R environment (<http://www.rstudio.com>).

One main feature of this software is that nearly all the user-computer communication is done through commands, in a specific programming language. This language is actually not much different than other languages such as those used in Splus and Matlab. So since you don't have menus and dialog boxes to run your analyses and computations from, you will have to learn the basics of the R programming language in order to perform data analysis.

<http://www.rseek.org/> is one of the best web search engines regarding R and its programming language.

2 The R General user interface (RGui)

When you open R, you start with a window called *RGui* which itself contains a window called *R Console*. The console is the place where all the numerical outputs are printed. You can actually run all your analyses and computations from the console, but once you close the software, everything that is printed in the console window is lost. Therefore we strongly recommend you to run everything from a script file that you can save and load.

A script is a text file with the *.R* extension in which you write commands meant to be sent to R. You create a script file through File -> New script. Notice that a script always opens in a new window, also embedded in the *RGui* window, which is called

Untitled until you save it through File -> Save as... and give it a name. You load a previously saved script through File -> Open script....

3 Data objects in R

Working in R means manipulating data objects, whether they are numerical variables, character strings or data files.

The main types of data objects are:

- **vector**: A vector of any dimension (including scalars).
- **matrix**: Vectors bound together.
- **data.frame**: Data object. More complicated than a matrix since it also contains variable names.
- **list**: Set of different objects, often outputs/results from analyses.
- **character**: Character string, usually found between quotes “ ”.

4 Basic Commands

To execute a command in the console, press Enter; to run a command from a script file, select it and type Ctrl+R. General remarks:

- **The prompt (>)**: In the console, every command (and output) is automatically displayed after the prompt > symbol; you don't have to write it yourself, the computer adds it automatically. There is no prompt in a script file.
- **Comments**: The # symbol is used for comments. On a code line, whatever is written after a # symbol is ignored by R when the command is executed. We recommend that you comment your script files as much as possible so that you don't get lost when opening them again.
- **Help**: If you need help regarding the syntax of a function, go to Help -> R functions (text)..., and type the name of the function. This is equivalent to type in the console `help()` with the name of the function between the parentheses.
- **Esc key**: If the console seems “stuck” or unresponsive, press Esc to get back to a new prompt. Be sure to check your syntax before sending the command again.

4.1 Create a variable/assignment

When you want to assign values to an object, you can either use the ‘<-’ symbol or the ‘=’ sign. You write the name of the object on the left of this symbol, the value you want to assign to it on the right.

For instance, the command `x <- 4` or `x = 4` assigns the value 4 to the object called *x*. If such object does not already exist, it is automatically created. In this particular example, *x* is considered as a vector of length 1.

To display the content of an object, type its name in the console and hit Enter.

Exercise: Create a new script file and save it in your H:\ directory. Next create an object called *y* and give it the value 10. Finally, check the content of *y* through the console.

You can create numerical vectors through the function `c()`. Between parentheses are the entries of this vector separated by commas.

Exercise: Create the *a* vector through `a <- c(2,4,6)`. Construct the *b* vector whose elements are 3, 5 and 7. Compute the sum of these two vectors by running `a+b`.

We can construct matrices by binding vectors by row or by columns. Try `rbind(a,b)` and `cbind(a,b)`. As you can see, `cbind` pastes columns together and `rbind` pastes rows.

4.2 Extracting elements from objects

To reach a specific element of a vector, you type its name and between square brackets `[]` you enter the number referring to the position of the element. For example, `a[2]` will give you back 4.

When dealing with matrices, you also use square brackets but identify the element by its row and column, the two numbers being separated by a comma.

Exercise: Create a matrix by binding the *a* and *b* vectors together through `cbind(a,b)`, call it *ab*. Then try `ab[1,2]`, `ab[,2]` and `ab[3,]`. How can you extract back the *a* vector from the *ab* matrix?

In a list object, you identify an element also by its position number but put between double square brackets `[[ ]]` or by typing the name of the list followed by the symbol `$` and the name of the element.

4.3 Operators

You can perform computations with the common mathematical operators: `+` addition, `-` subtraction, `*` multiplication, `/` division, `^` powers, `sqrt()` square root, `log()` natural logarithm and `exp()` exponential. Note that the operations priorities in general are the same as in mathematics, i.e. the parentheses have the highest priority, then power, then multiplication and division, and finally addition and subtraction.

Exercise: Run the following computations: $a = 3 * 5^2$, $b = (3 * 5)^2$, $c = e^{(3+5)}$,
 $d = \sqrt{9}$.

Exercise: Write the code for $z_1 = \frac{(a+b)c}{d}a$ and $z_2 = \frac{(a+b)c}{d^2}$.

4.4 Plots

Plots always appear in a separate window called *R Graphics*. You can copy/paste them, save them in many formats, etc. A few useful plot commands are:

<code>plot(x)</code>	General plot command, see <code>help(plot)</code> for details.
<code>boxplot(x)</code>	Constructs the boxplot of x .
<code>hist(x)</code>	Constructs the histogram of x .
<code>abline(...)</code>	Adds a straight line to a previously created plot. See <code>help(abline)</code> for details.
<code>lines(...)</code>	Adds a line to a previously created plot, using (x, y) coordinates. See <code>help(lines)</code> .
<code>win.graph()</code>	Opens a new empty <i>R Graphics</i> window.

4.5 Other useful commands

Here is a list of other basic commands and their effect:

<code>length(x)</code>	Gives back the dimension of x (x must be a vector).
<code>dim(x)</code>	Gives back the number of rows and columns of x (x must be a matrix).
<code>dimnames(x)</code>	Displays the names of the rows and columns of x .
<code>summary(x)</code>	Returns a descriptive summary of x , including the mean and quartiles.
<code>sum(x)</code>	Computes the sum of all the elements composing x .
<code>mean(x)</code>	Computes the arithmetic mean of the elements composing x .
<code>var(x)</code>	Computes the (bias-corrected) variance of the elements composing x .
<code>ls()</code>	Displays all the available objects in the environment.
<code>rm(x)</code>	Removes the x object from the R environment.

Exercise: Create the vector z with elements 1, 2, 3, 4 and 5. Compare `length(z)` and `dim(z)`.

5 Importing data

R can read many different formats of data, but as far as this course is concerned, you will only use the simplest formats such as text files with the *.txt* or *.dat* extensions. To import data in the R environment you use the `read.table(file.choose(),header=TRUE)` command. After sending the command, a dialog window pops up and you can browse your data file.

You set the optional argument `header` of the `read.table` function to `TRUE` if your data file features variable names in the first row, otherwise you write `header=FALSE` in the function.

6 Work with R and Rstudio

The most efficient way of working with R consists on saving the used commands in a script file for later execution. You can create it via the menu items **File -> New script**. To save all information coded in the file it is necessary to save the script with the extension *‘.R’*. While in the console is simply necessary to press **ENTER** to run your code, if you want to do it directly from the script you can use the option *run* in the menu or the command **CTRL+R** (**cmd+Enter** for Mac) after selecting the code you want to run.

In order to improve your experience of working with script files, you can use **Rstudio**, a very convenient interface providing code highlighting and easy interaction with the R environment. Moreover, with its system of *panes*, you can have viewers for the help files, plots, loaded packages in a single window.

Rstudio can be downloaded and installed for free via their website <https://www.rstudio.com>